

Three green apples are arranged in a cluster on a white background. One apple is in the foreground, slightly to the right, and two are behind it, one to the left and one to the right. The apples are bright green with some yellow highlights.

人工知能  
IE229 – ARTIFICIAL INTELLIGENCE

第11回講義: 画像認識  
Lecture 11: Image Recognition

二宮 崇 (Takashi Ninomiya)  
愛媛大学 (Ehime University)  
ninomiya.takashi.mk@ehime-u.ac.jp

第11回は画像認識について学びます。

# 今回の講義内容

- **ニューラルネットワークの学習**

- 確率的勾配降下法 (SGD)
- Momentum, AdaGrad, AdaDelta, Adam
- 過学習への対応: Weight Decay, Dropout, Batch Normalization

- **画像認識**

- 畳み込みニューラルネットワーク (Convolutional Neural Networks)



2

今回は画像認識について学びますが、まず、ニューラルネットワークの最適化手法について学びます。続いて、画像認識の基本となる畳み込みニューラルネットワーク (Convolutional Neural Networks; CNN)について学びます。

## 【復習】勾配 (gradient) (1/2)

- 全ての変数の偏微分をベクトルとしてまとめたもの

例： 関数 $L(w_0, w_1)$ の勾配

$$\nabla L = \left( \frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1} \right)$$

$L(w_0, w_1) = w_0^2 + w_1^2$ の $w_0 = 3$ 、 $w_1 = 4$ における勾配は $(6, 8)$

$$\nabla L = \left( \frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1} \right) = (2w_0, 2w_1)$$

$$\nabla L(3, 4) = (6, 8)$$



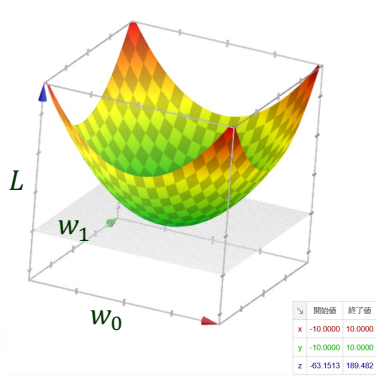
3

勾配は全ての変数の偏微分をベクトルとしてまとめたものです。例えば、関数 $L(w_0, w_1) = w_0^2 + w_1^2$ の勾配は $\nabla L = \left( \frac{\partial L}{\partial w_0}, \frac{\partial L}{\partial w_1} \right) = (2w_0, 2w_1)$ となります。従って、例えば、 $w_0 = 3$ 、 $w_1 = 4$ における勾配は $(6, 8)$ となります。

## 【復習】勾配 (gradient) (2/2)

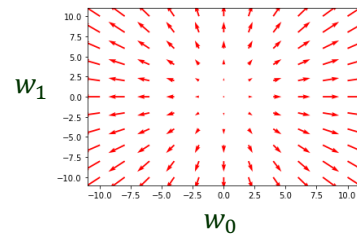
- その点における傾斜方向を表す

$$L(w_0, w_1) = w_0^2 + w_1^2$$

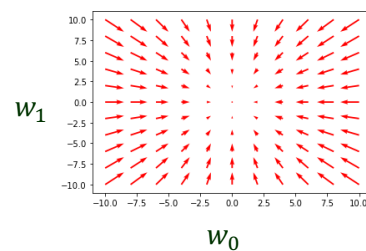


勾配に-1をかけたベクトルは各場所で関数の値を最も減らす方向を示す！！

各点における勾配



各点における勾配に-1をかけたベクトル



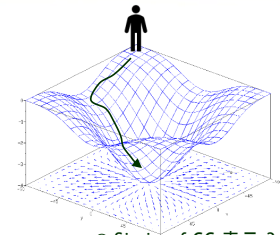
4

この $L(w_0, w_1) = w_0^2 + w_1^2$ の関数をグラフ化したものが左図となります。この関数に対する勾配に-1をかけたものは右図のようになっていて、中心に向かっていくことがわかります。また、中心から離れれば離れるほど勾配の大きさが大きいこともわかります。従って、どこの点からスタートしても、勾配の逆方向に進んでいくことで関数の最小値となるところにたどり着くことがなんとなくわかると思います。

## 【復習】勾配降下法 (Gradient Descent)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla L(\mathbf{w}) \quad \leftarrow \text{この計算を繰り返す}$$

$\eta$  : 学習率



© Simiprof CC 表示 3.0

ステップ0：初期値の設定

各パラメータの初期値を設定

ステップ1：勾配の算出

最小化する関数に基づき、パラメータの勾配を求める

ステップ2：パラメータの更新

パラメータを勾配方向とは逆向きに微小量更新する

ステップ3：繰り返す

ステップ1に戻る

$\nabla L(\mathbf{w})$

$-\eta \cdot \nabla L(\mathbf{w})$

5

勾配降下法の実体的なアルゴリズムはこのようになります。まず、各パラメータ(重み変数)の初期値を設定します(ステップ0)。

次にパラメータの勾配 $\nabla L(\mathbf{w})$ を求めます(ステップ1)。

続いて、パラメータを勾配方向と逆向きに微小量更新します。 $\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \nabla L(\mathbf{w})$  (ステップ2)

あとはこれをステップ1に戻って繰り返します。

# 確率的勾配降下法 (Stochastic Gradient Descent; SGD)

- ミニバッチ学習版の勾配降下法

- 勾配の計算に訓練データの一部を使用する

- NNのSGDによる学習

1. ミニバッチ：訓練データの中からランダムにミニバッチサイズ分のデータを選ぶ（もしくは訓練データの順番をシャッフルしてミニバッチサイズ分順番に取得する）
2. 勾配の算出：ミニバッチの損失に対する各重みパラメータの勾配を求める
3. パラメータの更新：重みパラメータを勾配方向とは逆方向に微小量更新する
4. 繰り返す：ステップ1に戻る

$$w = w - \eta \frac{\partial L}{\partial w}$$

6

勾配降下法は全データに対する勾配を計算して最適化する手法ですが、一部のデータをサンプリングすることにより勾配を近似的に計算して最適化する手法は確率的勾配降下法と呼ばれます。

訓練データの中からランダムに選んだデータはミニバッチと呼ばれます。ミニバッチの大きさはミニバッチサイズと呼ばれます。ミニバッチの損失に対する各重みパラメータの勾配を求め、得られた勾配方向と逆方向にパラメータを微小量更新します。アルゴリズムとしては、次のようになります。

## NNのSGDによる学習

ミニバッチ：訓練データの中からランダムにミニバッチサイズ分のデータを選ぶ(もしくは訓練データの順番をシャッフルしてミニバッチサイズ分順番に取得する)

勾配の算出：ミニバッチの損失に対する各重みパラメータの勾配を求める

パラメータの更新：重みパラメータを勾配方向とは逆方向に微小量更新する

繰り返す：ステップ1に戻る

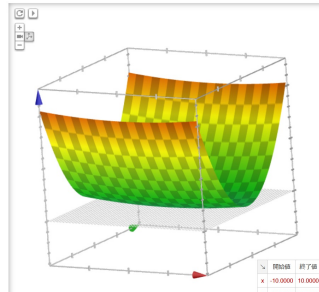
# SGDの欠点

- 勾配の方向が本来の最小値ではない方向を指しているときは非効率な場合がある

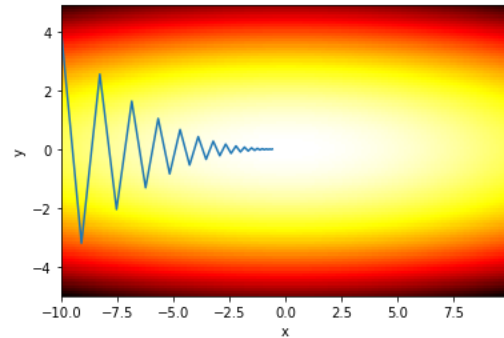
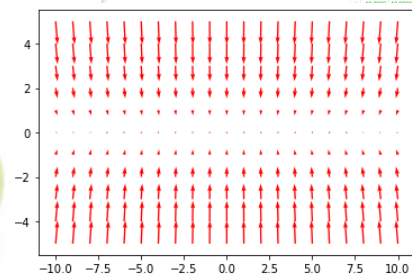
例：

$$f(x, y) = \frac{1}{20}x^2 + y^2$$

最小値(0,0)



-1×勾配



$$\mathbf{w} = \mathbf{w} - \eta \cdot \frac{\partial L}{\partial \mathbf{w}}$$

ジグザグで最小値に近づく  
→非効率

7

SGDは単純ながら非常に良い学習を行います。

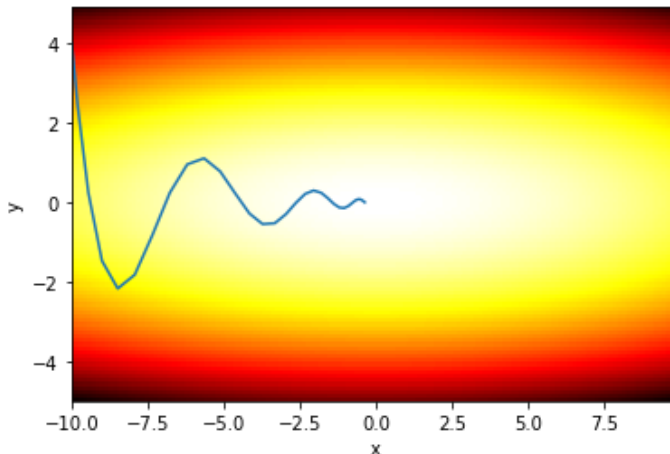
しかしながら、SGDの欠点として、勾配の方向が本来の最小値ではない方向を指していると非効率になる場合があります。例として、 $f(x, y) = \frac{1}{20}x^2 + y^2$ の空間で $f(x, y)$ が最小となる $x, y$ を求めることを考えてみます。 $f(x, y)$ の空間は左上の図のようになりますが、谷のようになっていて、谷底に緩やかな傾斜がついて、(0,0)が最小値となるようになっています。その下の図は $f(x, y)$ の勾配の逆方向を示しています。谷のようになっていて、谷底に向かってほぼ垂直に勾配の逆方向が向かっています。

このような空間で勾配に従って学習を行うと、右の図のようにジグザグで最小値に向かって近づくため学習効率が悪くなります。つまり、谷を飛び越える形でパラメータが推移しています。

# 最適化手法：Momentum

$$v = \alpha v - \eta \cdot \frac{\partial L}{\partial w}$$
$$w = w + v$$

$v$  : 速度の役割  
(勾配方向に速度が加算される)  
 $\alpha$  : ハイパーパラメータ(0.9等)



- x軸の勾配の方向は一定  
→x軸方向には加速される
- y軸の勾配の方向は交互  
→y軸方向への動きは抑制  
⇒ジグザグの動きが軽減

8

SGDを改良するためにいくつかの手法が提案されています。その一つがMomentumです。

Momentumは、SGDの学習に速度の概念が追加されたアルゴリズムとなっています。 $v = \alpha v - \eta \cdot \frac{\partial L}{\partial w}$ によって、 $v$ を計算しますが、この $v$ が速度の役割を果たします。勾配が加速度の役割を果たしています。 $\alpha$ と $\eta$ がハイパーパラメータ(訓練データ以外のデータで調整しないといけないパラメータ)となります。ハイパーパラメータは、開発データ (development set)もしくは検証データ (validation set)と呼ばれるテストデータでの性能をみて、手で調整することが一般的です。ただし、開発データ(検証データ)はテストデータとは異なる別のデータとなっていなければなりません。

この速度のおかげで、x軸方向には加速され、y軸方向は動きが抑制されることになります。このため、ジグザグの動きが軽減され滑らかに学習されることがわかります。

# 最適化手法： AdaGrad

- パラメータ毎に学習率をコントロール

- 最初は大きく次第に小さく
- 大きく更新されたパラメータほど小さくする

$$\begin{aligned} \mathbf{h} &= \mathbf{h} + \frac{\partial L}{\partial \mathbf{w}} \otimes \frac{\partial L}{\partial \mathbf{w}} \\ \mathbf{w} &= \mathbf{w} - \eta \frac{1}{\sqrt{\mathbf{h}}} \otimes \frac{\partial L}{\partial \mathbf{w}} \end{aligned}$$

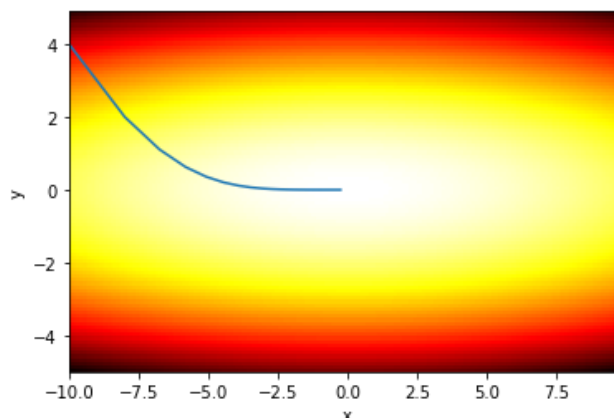
⊗: 要素毎の積

y軸方向の勾配は大きい

→最初は大きく更新されるが

更新が進むほど更新度合いが弱まる

⇒ジグザグの動きが軽減



9

SGDを改良するもう一つの手法にAdaGradと呼ばれる最適化手法があります。

AdaGradではパラメータ毎に学習率をコントロールします。パラメータは次の式に従って更新されます。

$$\begin{aligned} \mathbf{h} &= \mathbf{h} + \frac{\partial L}{\partial \mathbf{w}} \otimes \frac{\partial L}{\partial \mathbf{w}} \\ \mathbf{w} &= \mathbf{w} - \eta \frac{1}{\sqrt{\mathbf{h}}} \otimes \frac{\partial L}{\partial \mathbf{w}} \end{aligned}$$

ただし、⊗は要素毎の積を表します。 $\mathbf{h}$ は、勾配の各要素を2乗した値を足し続けることによって、過去に各パラメータがどの程度更新されたかを記憶しています。 $\frac{1}{\sqrt{\mathbf{h}}} \otimes \frac{\partial L}{\partial \mathbf{w}}$ では、 $\mathbf{h}$ の平方根の逆数がかけられることによって、すでに大きく更新されたパラメータの勾配が小さくなるようになっています。つまり、すでに大きく更新されたパラメータほど小さく更新されるようになります。この手法によって、各パラメータは最初は大きく次第に小さく更新されるようになります。

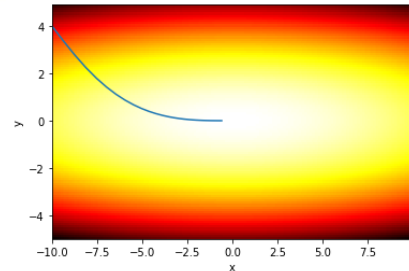
先程の例をみると、y軸の勾配が大きいので最初大きく更新されるのですが、すぐに小さくなります。このためジグザグの動きが軽減されます。

## その他の最適化手法

- Adadelata

- AdaGradの発展形

※ Matthew Zeiler, ADADELTA: An Adaptive Learning Rate Method, 2012.

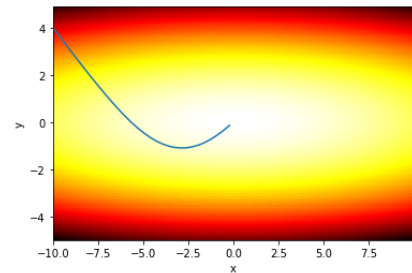


- Adam

- AdaGradのアイデア + Momentumのアイデアの融合

※ Diederik Kingma and Jimmy Ba, Adam: a Method for Stochastic Optimization, 2015.

など



10

その他の最適化手法として、AdadelataとAdamがあります。AdadelataはAdaGradの発展形となります。AdamはAdaGradとMomentumのアイデアを融合したものとなります。実際のニューラルネットワークの学習では、AdamかSGDのどちらかがよく用いられています。

# パラメータの初期値

- 活性化関数がシグモイド関数の場合

- Xavierの初期値： $\frac{1}{\sqrt{n}}$ を標準偏差とするガウス分布で各パラメータを初期化（ $n$ ：前層のノード数）

Xavier Glorot and Yoshua Bengio, Understanding the difficulty of training deep feedforward neural networks, 2010.

- 活性化関数がReLUの場合

- Heの初期値： $\frac{2}{\sqrt{n}}$ を標準偏差とするガウス分布で各パラメータを初期化（ $n$ ：前層のノード数）

Kaiming He, Xiangyu Zhang, Shaoqing Re, and Jian Sun, Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification, 2015.



11

各パラメータの初期値も非常に重要です。活性化関数がシグモイド関数の場合、Xavierの初期値がよく用いられます。活性化関数がReLUの場合はHeの初期値がよく用いられます。

Xavierの初期値は、 $\frac{1}{\sqrt{n}}$ を標準偏差とするガウス分布で各パラメータを初期化します。ただし、 $n$ は前層のノード数です。

Heの初期値は、 $\frac{2}{\sqrt{n}}$ を標準偏差とするガウス分布で各パラメータを初期化します。ただし、 $n$ は前層のノード数です。

## 勾配爆発を防ぐ: 勾配クリッピング

- 勾配爆発を防ぐために勾配が大きすぎる場合に強制的に小さくする

- 1. 勾配の大きさ  $\|\nabla L\|$  を計算
- 2. ある閾値  $\theta$  に対し、勾配の大きさ  $\|\nabla L\|$  が  $\theta$  を超えていたら、勾配をスケールする

$$\nabla L \leftarrow \frac{\theta}{\|\nabla L\|} \nabla L$$



12

学習時に勾配が大きくなりすぎて学習がうまくいかないことを勾配爆発と呼びます。勾配爆発を防ぐために勾配クリッピングという方法が使われています。

アルゴリズム的には次のようになります。

1. 勾配の大きさ  $\|\nabla L\|$  を計算
2. ある閾値  $\theta$  に対し、勾配の大きさ  $\|\nabla L\|$  が  $\theta$  を超えていたら、勾配をスケールする

$$\nabla L \leftarrow \frac{\theta}{\|\nabla L\|} \nabla L$$

# 過学習への対応法：荷重減衰（Weight Decay）

- 重みのL2ノルムを正則化項として加えた損失に基づき学習

- Weight Decayを用いた損失

$$L(\mathbf{w}) + \frac{1}{2}\lambda\|\mathbf{w}\|_2^2$$

$\lambda$ ：ハイパーパラメータ

$\mathbf{w} = (w_1, w_2, \dots, w_n)$ に対するL2ノルムは

$$\|\mathbf{w}\|_2 = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

従って、

$$\|\mathbf{w}\|_2^2 = w_1^2 + w_2^2 + \dots + w_n^2$$



13

次にニューラルネットワークにおける過学習への対応法について紹介します。  
最初は荷重減衰 (Weight Decay) と呼ばれる手法です。これは機械学習における正則化と同じものです。Weight Decayを用いた損失は次のようになります。

$$L(\mathbf{w}) + \frac{1}{2}\lambda\|\mathbf{w}\|_2^2$$

$\lambda$ はハイパーパラメータです。 $\mathbf{w} = (w_1, w_2, \dots, w_n)$ に対するL2ノルムは

$$\|\mathbf{w}\|_2 = \sqrt{w_1^2 + w_2^2 + \dots + w_n^2}$$

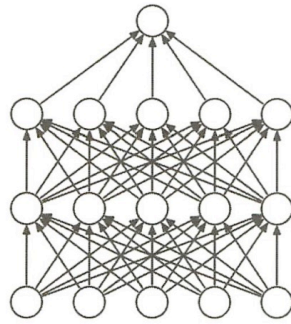
従って、

$$\|\mathbf{w}\|_2^2 = w_1^2 + w_2^2 + \dots + w_n^2$$

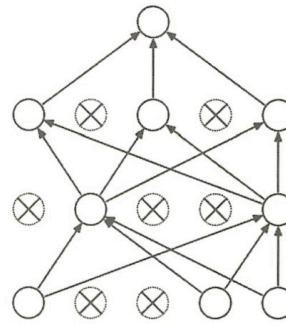
となります。

# 過学習への対応: Dropout

- 学習時にはデータが流れるたびに中間層のノードをある割合(dropout率)でランダムに選択し消去して学習
- テスト時には各層の出力に対して訓練時に消去した割合(dropout率)を乗算した値を出力



Dropoutなし



Dropoutあり



N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, Dropout: A simple way to prevent neural networks from overfitting, 2014.

14

過学習への対応法として次にDropoutを紹介します。

Dropoutは、学習時に中間層のノードをランダムに消去して学習する手法のことです。学習時、データが流れるたびに中間層のノードをある割合(dropout率)でランダムに選択し消去して学習します。テスト時には、各層の出力に対して、訓練時に消去した割合(dropout率)を乗算した値を出力します。

# 過学習への対応: Batch Normalization

- ミニバッチ毎に入力データを正規化

ミニバッチ:  $\{x_1, x_2, \dots, x_m\}$   
 $\varepsilon$ : 小さな値(10e-7等)

$$\begin{aligned}\mu &= \frac{1}{m} \sum_{i=1}^m x_i \\ \sigma^2 &= \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2 \\ \hat{x}_i &= \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}\end{aligned}$$

- 効果

- 学習が速くなる
- 初期値への依存度を減らせる
- 過学習を抑制する

 Sergey Ioffe and Christian Szegedy, Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift, 2015.

15

過学習への対応法として最後にBatch Normalizationを紹介します。

Batch Normalizationはミニバッチ毎に入力データを正規化する手法です。ミニバッチ内での入力を $x_1, x_2, \dots, x_m$ としたとき、

$$\begin{aligned}\mu &= \frac{1}{m} \sum_{i=1}^m x_i \\ \sigma^2 &= \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2 \\ \hat{x}_i &= \frac{x_i - \mu}{\sqrt{\sigma^2 + \varepsilon}}\end{aligned}$$

を計算し、 $\hat{x}_i$ を入力とする手法です。つまり、ミニバッチ内で入力の値を正規化している、と言えます。

Batch Normalizationによって、学習が速くなる、初期値への依存度を減らせる、過学習を抑制する、といった効果があると言われています。

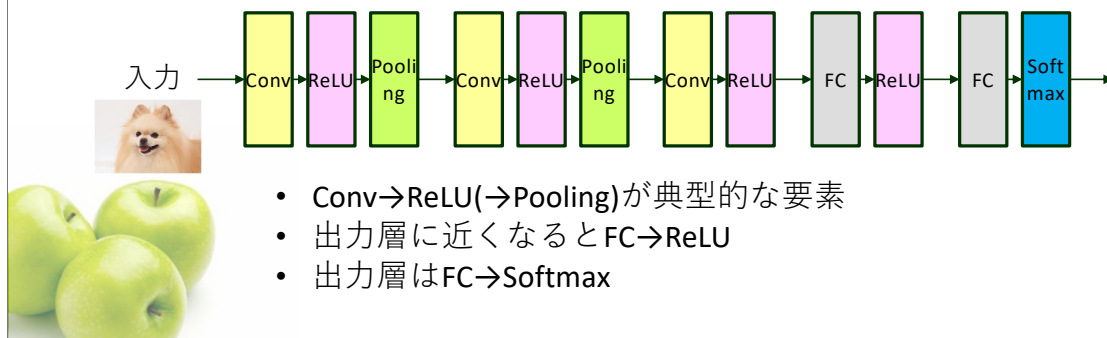
# 畳み込みニューラルネットワーク (CONVOLUTIONAL NEURAL NETWORK; CNN)



# 畳み込みニューラルネットワーク (Convolutional Neural Network; CNN)

- 入力の形状／構造を捉えることに適したネットワーク
  - 特に画像認識には有効なNN
- 畳み込み層(Convolution Layer)とプーリング層(Pooling Layer)を持つ

一般的なCNN



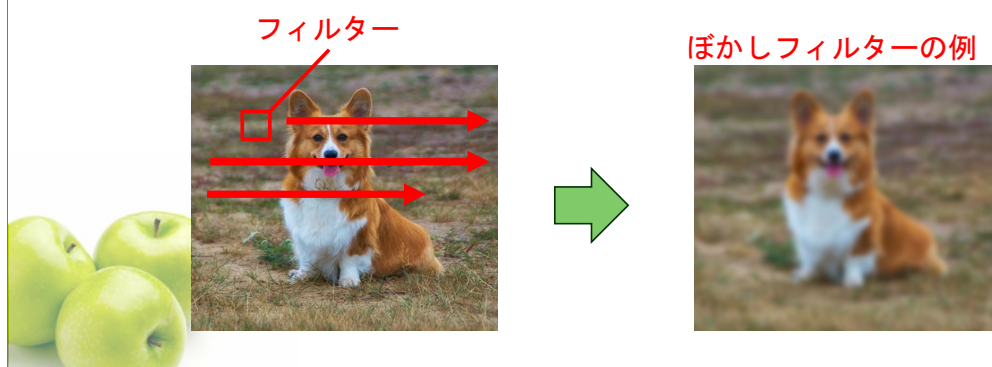
17

畳み込みニューラルネットワーク(Convolutional Neural Network; CNN)は、入力の形状および構造を捉えることに適したネットワークで、特に画像認識に有効なNNとして知られています。CNNは畳み込み層(Convolution Layer)とプーリング層(Pooling Layer)を繰り返し持つ構造のニューラルネットワークのことです。Conv→ReLU(→Pooling)が典型的な構成要素となります。出力層に近くなると、FC→ReLUとなり、画像分類の場合は最後にFC→Softmaxとなります。従って、畳み込み層とプーリング層が行うことを理解すればCNNを理解したことになります。

# 畳み込み層

## ● 重み付きフィルターによる画像処理

- 全ての画素に対しフィルターをかけて画像全体になんらかの効果を与える
- フィルターには重みがついていて、周りの画素情報から中心の画素の計算を行う
- 通常のフィルターは人手で与えたものを用いるが、CNNではフィルターを自動的に学習する



18

畳み込み層では、重み付きフィルターによる画像処理を行います。

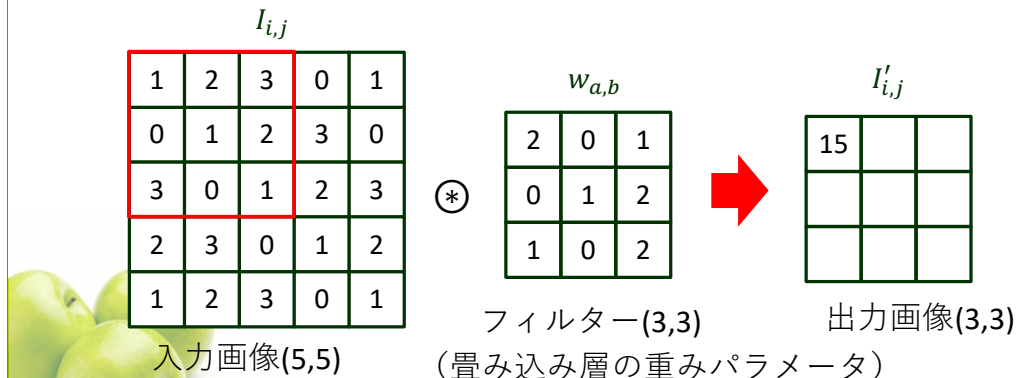
全ての画素に対し、フィルターをかけて画像全体に何らかの効果を与えることを行います。イメージ的には、左上の画素からスタートして、全ての画素を走査するように処理を行うと考えてもらえば良いです。(GPUを用いる場合はこれらの処理を並列に計算できる)。フィルターには重みがついていて、周りの画素情報からフィルターがかけられている中心の画素の計算を行います。一般的な画像処理では、人手で定義したフィルターを使って、画像をぼかしたり、エッジを抽出することが行われています。スライドの図はぼかしフィルターの例になります。

CNNでは、このフィルターの重みを自動的に学習することを行います。

# 畳み込み層

- 画像の場合、3次元テンソル(縦、横、チャンネル)のデータを入力として受け取り3次元テンソル(縦、横、チャンネル)を出力
- フィルターによる畳み込み演算 (3x3のフィルターの場合)

$$I'_{i,j} = \sum_{a=-1}^1 \sum_{b=-1}^1 w_{a,b} I_{i+a,j+b}$$



19

具体的にどのように計算するのかみていきましょう。畳み込み層は、縦、横、チャンネルの3次元テンソルに対し処理を行います。縦と横で2次元の画像を表していて、チャンネルは特徴ベクトルを表していると考えれば良いです。つまり、各画素ごとにチャンネル数と同じ次元数のベクトルがついているということです。一番最初の入力となる画像に対しては、通常カラー画像なので、RGBの3次元ベクトルが各画素の特徴ベクトルとなっているということです。つまり、最初の入力画像はチャンネル数3となります。

各画像に対して畳み込み演算を行っていくのですが、まず、1チャンネルの画像に対してどのように処理を行うのかみてみましょう。入力画像(I)に対して、フィルター(w)を適用して、出力画像I'を得ます。この例では入力画像が5x5のサイズの画像で、フィルターは3x3の大きさとしします。この場合、出力は3x3の画像になります。出力画像の各画素 $I'_{i,j}$ は、この例の場合(フィルター3x3の場合)、

$$I'_{i,j} = \sum_{a=-1}^1 \sum_{b=-1}^1 w_{a,b} I_{i+a,j+b}$$

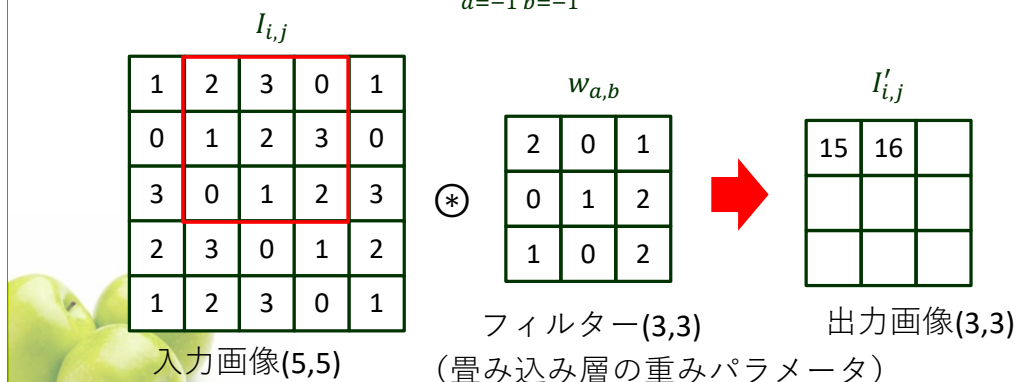
として、計算します。前のスライドで説明したようにフィルターを各画素に適用することで畳み込み演算が行われるのですが、フィルターを適用している場所(ウィンドウ)の各画素の値と対応するフィルターの各値をかけ合わせてウィンドウ内の全ての値を足したものをそのフィルターの中心の画素の値とします。例をみてみましょう。今スライドの赤枠のところにフィルターを適用しようとしています。この赤枠の中心部分の画素値を新しく計算しようとしています。このとき、赤枠内の各画素とそれに対応するフィルターの値をかけて、全て足し合わせることでこの赤枠の中心部分の値が計算されます。赤枠内の左上からみていくと、まず、 $1(I_{1,1}) \times 2(w_{1,1})$ を計算して2となります。次に $2(I_{1,2}) \times 0(w_{1,2}) = 0$ となります。同様に各画素の値とフィルターの値をかけていきます。最後にこれらの値を全部足し合わせます。すると、 $1 \times 2 + 2 \times 0 + 3 \times 1 + 0 \times 0 + 1 \times 1 + 2 \times 2 + 3 \times 1 + 0 \times 0 + 1 \times 2 = 15$ となりますので、出力の一番左上の値は15となります。 $I'_{2,2} = 15$ となります。

# 畳み込み層

- データの次元数を維持する

- 画像の場合、3次元（縦、横、チャンネル）のデータを入力として受け取り3次元のデータを出力して次の層に渡す
- フィルタによる畳み込み演算を行う（3x3のフィルタの場合）

$$I'_{i,j} = \sum_{a=-1}^1 \sum_{b=-1}^1 w_{a,b} I_{i+a,j+b}$$



20

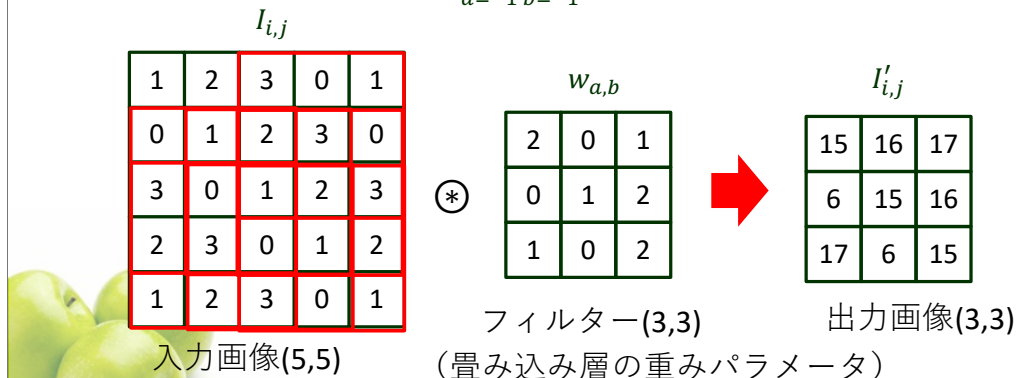
次のフィルタを1画素分右に移動させて、同様の計算を行います。今度は、 $2(I_{1,2}) \times 2(w_{1,1})$ を計算して4となります。次に $3(I_{1,3}) \times 0(w_{1,2})=0$ となります。同様に各画素の値とフィルタの値をかけて足し合わせます。すると、 $2 \times 2 + 3 \times 0 + 0 \times 1 + 1 \times 0 + 2 \times 1 + 3 \times 2 + 0 \times 1 + 1 \times 0 + 2 \times 2 = 16$ となりますので、出力の一番上真ん中の値は16となります。 $I'_{2,3} = 16$ となります。

# 畳み込み層

- データの次元数を維持する

- 画像の場合、3次元（縦、横、チャンネル）のデータを入力として受け取り3次元のデータを出力して次の層に渡す
- フィルタによる畳み込み演算を行う（3x3のフィルタの場合）

$$I'_{i,j} = \sum_{a=-1}^1 \sum_{b=-1}^1 w_{a,b} I_{i+a,j+b}$$



21

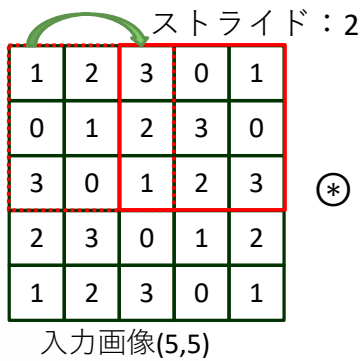
同様にフィルタを右に一つずらして、出力画像の右上の値を求めます(計算結果は17)。続いて、入力画像の右端にフィルタが到着したので、1画素下に移動して、左端から同様の計算を行います。

最終的に入力画像の一番右下にフィルタが到着すればこの画像に対する畳み込み演算はおわりになります。

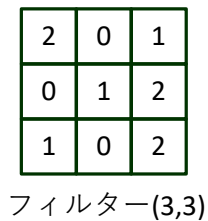
画像の周辺画素に対しては、フィルタを適用できないので、出力画像は周辺1画素分少なくなります。畳み込み演算を行う度に画像が小さくなっていく問題はパディングという手法で回避することができます。後でパディングについて説明します。

# ストライド

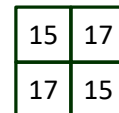
- ストライド: フィルターの移動幅



(\*)



※ストライドを増やすと  
出力データのサイズが減る



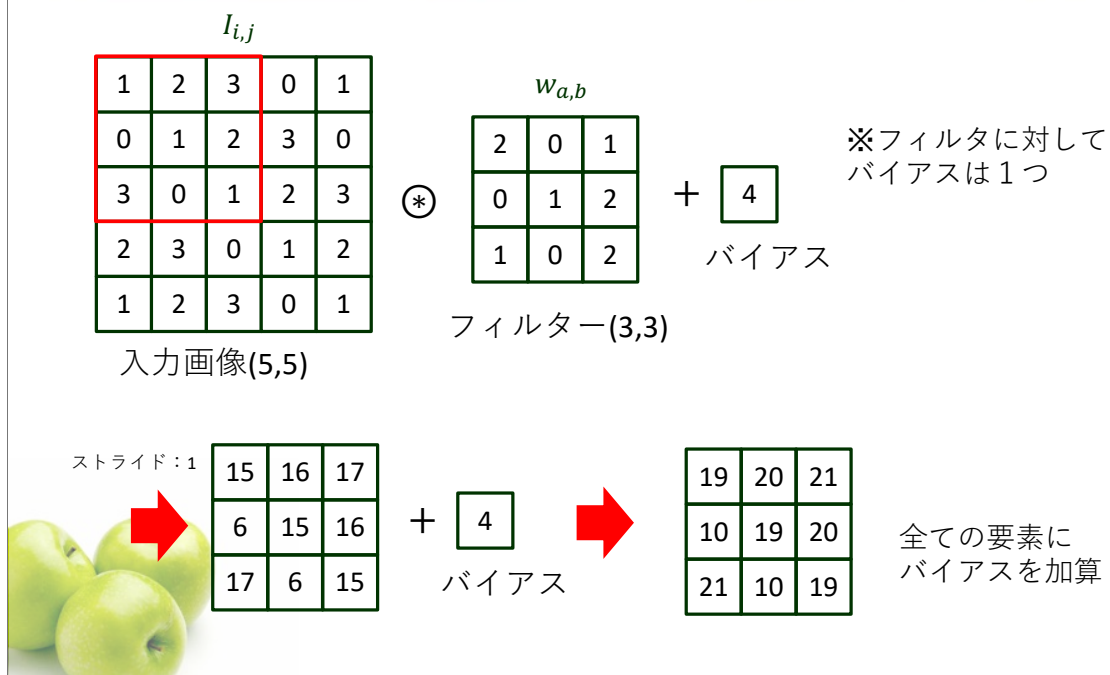
出力画像(2,2)



22

畳み込み演算を行う際にフィルターを1画素ずつずらしながら適用しましたが、複数画素ずらして適用することもできます。このフィルターの移動幅のことをストライドと言います。前のスライドではストライド数が1でした。このスライドの例はストライド数2の例となっています。フィルターを2画素ずつずらして畳み込み演算を行います。ストライド数を大きくすると、出力画像がかなり小さくなるのがわかります。(ストライド数を2にすると出力画像は入力画像のおおよそ1/4になります)

# 畳み込み演算にバイアスを導入



23

続いて、畳込み演算にバイアスを導入します。先程と同じ様に入力画像に対して、フィルタを適用したあと、その出力画像に対して、各画素にバイアスを足しこみます。このスライドの例だと、まず、入力画像にフィルタをかけることで、

15, 16, 17  
 6, 15, 16  
 17, 6, 15

の画像が得られます。これにバイアス4をたすと、それぞれの画素値に対して、4を足すことになるので、

19, 20, 21  
 10, 19, 20,  
 21, 10, 19

が最終的な出力となります。

# パディング

- 畳み込み層の処理をする度に画像の周辺が1画素ずつ削れていく(小さい画像や深いNNであるときに問題)
- **パディング**: 入力データの周囲を固定データ(例えば0)で埋めること

幅1のパディング

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 2 | 3 | 0 | 0 |
| 0 | 0 | 1 | 2 | 3 | 0 |
| 0 | 3 | 0 | 1 | 2 | 0 |
| 0 | 2 | 3 | 0 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 |

入力画像(4+2,4+2)

(\*)

|   |   |   |
|---|---|---|
| 2 | 0 | 1 |
| 0 | 1 | 2 |
| 1 | 0 | 2 |

フィルター(3,3)

ストライド:1



|    |    |    |    |
|----|----|----|----|
| 7  | 12 | 10 | 2  |
| 4  | 15 | 16 | 10 |
| 10 | 6  | 15 | 6  |
| 8  | 10 | 4  | 3  |

出力画像(4,4)

パディングによって入力画像と出力画像のサイズが同じになる

24

畳み込み演算を行う度に画像の周辺が1画素ずつ削れていくという問題について述べましたが、それに対応する手法がパディングと呼ばれる手法です。1画素ぐらい削れても問題無いような気がしますが、CNNの中では処理が進むにつれて画像サイズが小さくなるため、周辺1画素が削られるとかなりの情報を捨ててしまうことになります。また、数十層、数百層あるような深いCNNになっている場合は、その都度周辺が削れられると画像の大部分が削れてしまうことになります。

パディングは入力データの周囲を固定データ(例えば0)で埋める手法のことです。例えば、4x4の入力画像があったとき、この画像の周辺に0を埋めて6x6の画像にします。これに対して、ストライド1のフィルターをかけると、出力は4x4になって、結果、入力画像と出力画像のサイズが同じになります。これによって画像がどんどん削れていく問題を回避することができます。

## 【参考】 入力サイズと出力サイズの関係

入力データのサイズ  $(H, W)$ 、フィルターサイズ  $(FH, FW)$   
ストライド  $S$ 、パディングの幅  $P$  の時

出力データのサイズ  $(OH, OW)$  は

$$OH = \frac{H + 2P - FH}{S} + 1, \quad OW = \frac{W + 2P - FW}{S} + 1$$



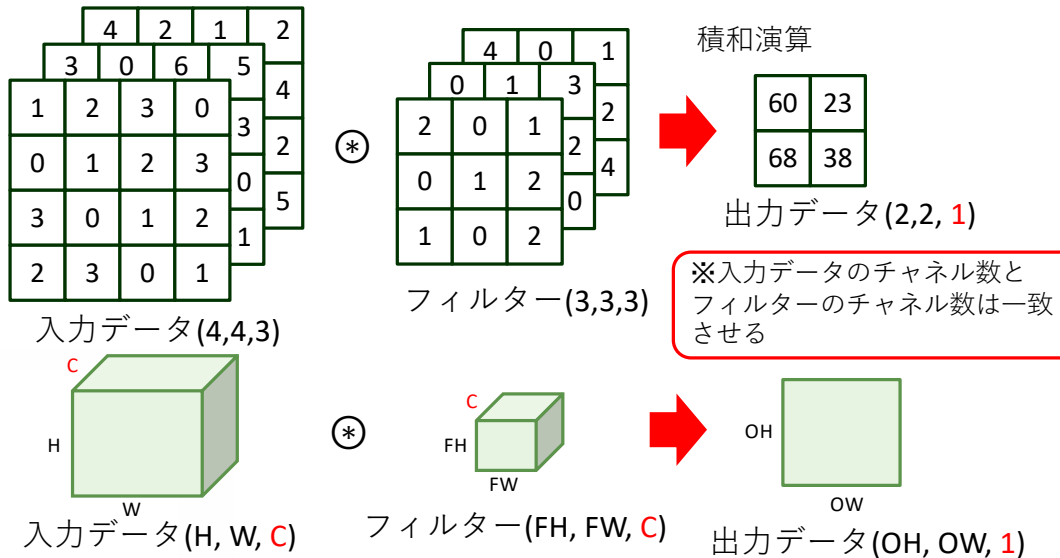
25

参考までに、入力データ、フィルターサイズ、ストライド、パディング幅に対する出力データのサイズの計算式を紹介しておきます。

## 3次元データの畳み込み演算(1/2)

### ● 画像データは3次元データ(縦、横、チャンネル)

3次元データの畳み込み演算 (ストライド: 1)



次に複数チャンネルある場合の畳み込み演算について説明します。一般的に画像は縦、横、チャンネルの3次元配列(テンソル)で入力されます。ここでは、チャンネル数3の画像データに対する畳み込み演算についてみてみます。それぞれ、R(赤)、G(緑)、B(青)の画像と思えば良いです。

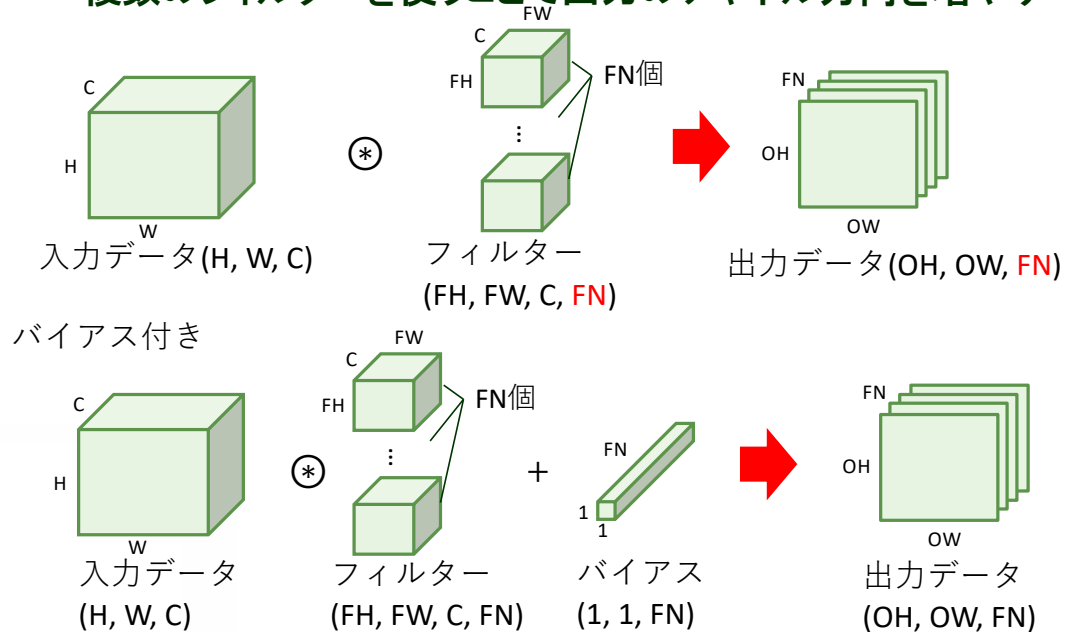
まず、畳み込み演算を行う画像のチャンネル数と同数のフィルタを用意します。つまりこの場合、3枚のフィルタを用意します。各チャンネルの画像に対し、それぞれに対応するフィルタを用いて先程説明した畳み込み演算を行います。そして、各チャンネルで計算された出力画素値を全てたしあわせて、出力画像の各画素の値とします。

一般に畳み込み層の入力出力は画像とは限らないので、入力データ、出力データと呼ぶことにします。今、入力データは4x4の行列が3チャンネルあるので、4x4x3のテンソルとなっています。これを(4, 4, 3)と表示します。続いて、フィルタは3x3の行列がやはり3チャンネルあるので、3x3x3のテンソルとなります。出力データはこれらを足しあわせて1枚の行列になるので、2x2x1のテンソルとなります。

一般に深層学習の世界では、テンソルを箱の形で表すことが多いです。入力データは高さ(H)x幅(W)xチャンネル(C)の箱で表現されて、フィルタもFHxFWxCの箱で表現されます。出力は1枚の行列になるので、OWxOWの四角となります。

## 3次元データの畳み込み演算(2/2)

- 複数のフィルターを使うことで出力のチャンネル方向を増やす

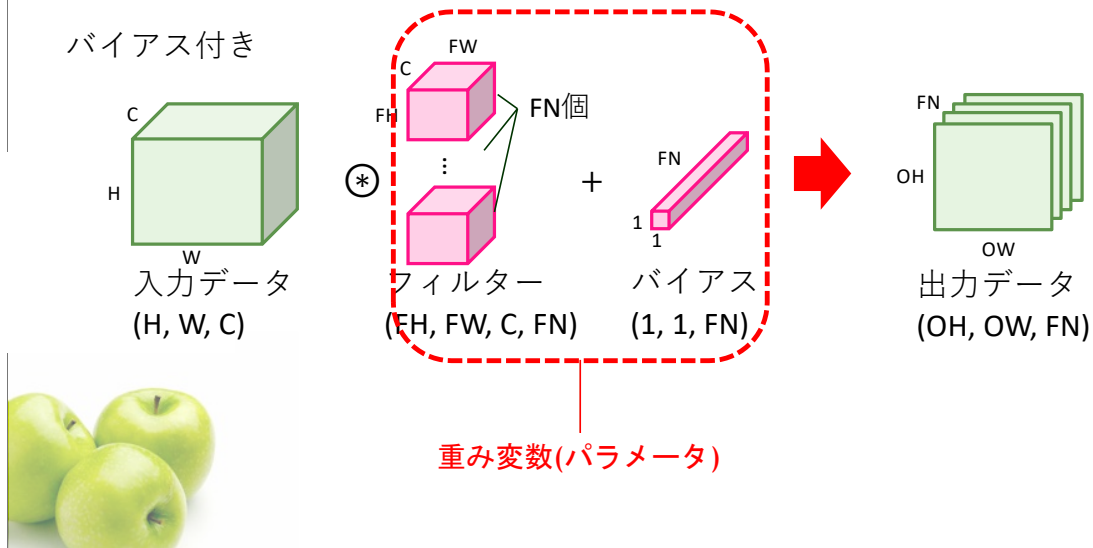


複数枚のフィルターを一つの箱で表現しましたが、これらの箱を複数個用意することで、出力を多次元化することができます。入力データを $H \times W \times C$ の大きさの箱だとします。フィルターは $FH \times FW \times C$ の箱で表現されていましたが、これが $FN$ 個あるとします。つまりフィルターは $FH \times FW \times C \times FN$ の4次元テンソルとなります。そうすると、出力は、 $OH \times OW \times FN$ の3次元テンソルとなります。例えば、入力が $H \times W \times 3$ のRGB画像だとすると、これに畳み込み層を適用して、例えば、 $H \times W \times 100$ の出力を得ることもできます。つまり各画素の特徴が3つ(3次元ベクトル)だったものが、100個の特徴(100次元ベクトル)になるというわけです。

続いて、これにバイアスをつけることを考えます。バイアスは出力データの各画素に対して足し込むので、 $(1 \times 1 \times FN)$ のテンソルで表現します。出力データの各行列に対してバイアス値が一つずつある、ということになります。

## 畳み込み層の重み変数(パラメータ)

- フィルターとバイアスが重み変数(パラメータ)



畳み込み層の例では具体的な数値が入ったフィルターを用いて説明しましたが、このフィルターとバイアスの各値は重み変数(パラメータ)となっていて、CNNによる画像認識の学習の過程で自動的に学習されるということに注意してください。

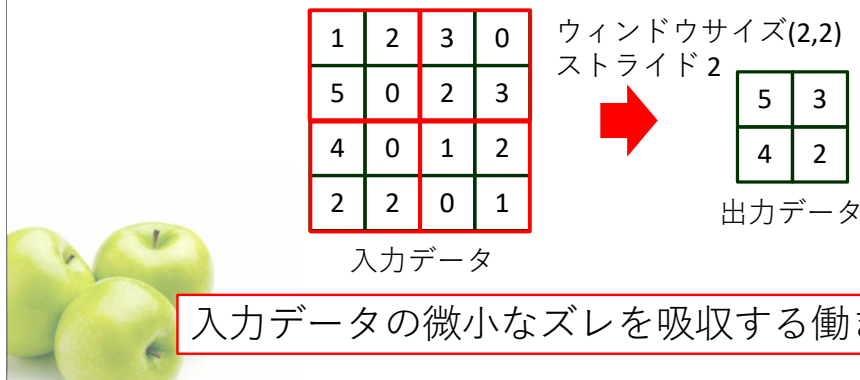
# プーリング層

- 縦・横方向の空間を小さくする演算(ダウンサンプリング)

- 学習パラメータはない
- チャンネル数は変化しない

- 最大(Max)プーリング

- 領域内の最大の要素を取り出す



29

畳み込み層の説明は以上で、次はプーリング層の説明になります。畳み込み演算に比べるとプーリング層は比較的単純な操作をやっています。

プーリング層は縦・横方向の空間を小さくする演算です。つまり、画像のダウンサンプリングを行う処理となります。学習パラメータは存在せず、チャンネル数も変化しません。

有名なプーリングの一つに最大プーリング(max pooling)があります。画像内の各領域においてその最大値でその領域の代表値とする手法です。スライドの図のような入力データがあったとき、これに対して、2x2のウィンドウサイズ、ストライド数2の最大プーリングを適用してみます。ストライド2なので、フィルターと同様に2つずつずらしながら最大プーリングを適用します。最大プーリングを適用する領域の大きさのことをウィンドウサイズ(プーリングサイズ)と言います。フィルターサイズと同じようなものと思って良いです。この領域内で最大の値をその領域の代表値とします。この場合、まず、左上の領域では、1,2,5,0の値があるので、最大値の5を左上の領域の代表値とします。続いて右上の領域では、3,0,2,3の値があるので、最大値である3をこの領域の代表値とします。同様に、左下では、4,0,2,2の中で最大となる4を代表値とします。最後に右下の1,2,0,1に対しては、その最大値2をこの領域の代表値とします。

一般にプーリング層は画像を小さくする(ダウンサンプリング)ために用いられています。画像を小さくすることで、入力データの微小なズレを吸収する働きがあると言われています。

# Maxプーリング以外のプーリング演算

- 平均(Average)プーリング

- 領域内の平均を各要素とする

入力データ

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 0 |
| 5 | 0 | 2 | 3 |
| 4 | 0 | 1 | 2 |
| 2 | 2 | 0 | 1 |

ウィンドウサイズ (2,2)  
ストライド 2



|   |   |
|---|---|
| 2 | 2 |
| 2 | 1 |

出力データ

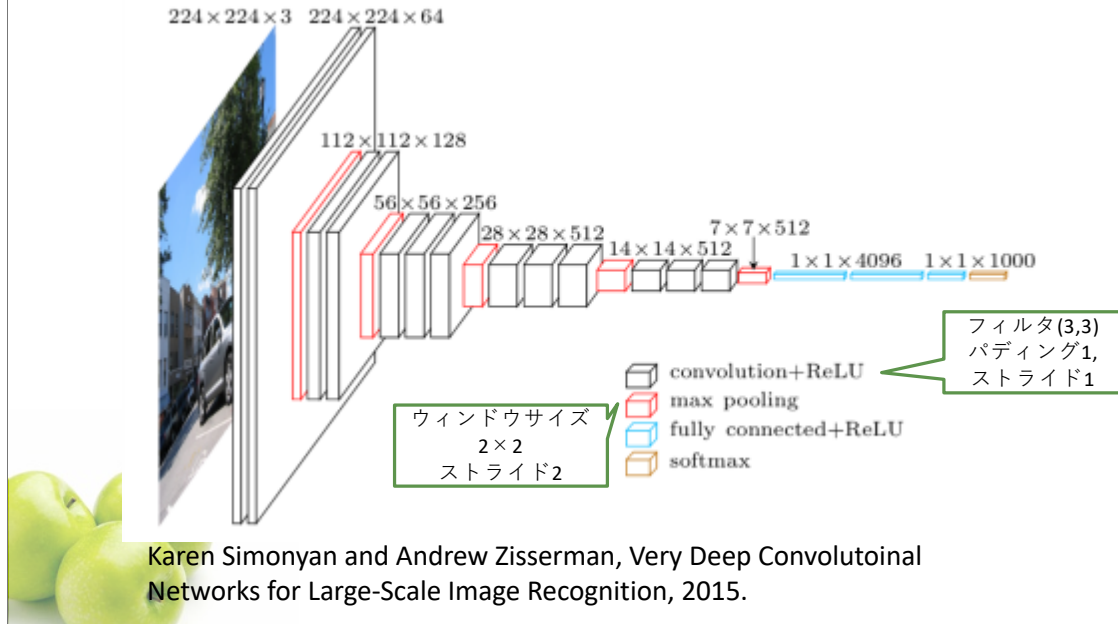


30

最大プーリングの他に有名なプーリング層として、平均プーリング(Average Pooling)があります。最大値をとる代わりに、平均値をその領域の代表値としています。

# 代表的なCNN：VGG16

## ● 畳み込み層 13層+FC層 3層



31

CNNにはいろんなタイプのものがありますが、比較的簡単な構造で性能が高いものとして、VGG16と呼ばれているCNNが有名です。VGG16は畳み込み層13層とFC層3層から成るCNNで、スライドの図のような構成になっています。

最初は224x224のRGB3枚の画像データが入力として用いられています。まず、畳み込み層+ReLUが2層あり、224x224x64のデータに変換されます。3色の特徴が64次元の特徴に変換されていることに注意してください。

続いて、最大プーリングを用いて、画像サイズを縦横それぞれ1/2にします(112x112x64)。続いて、畳み込み層+ReLUを2層通して112x112x128のデータに変換しています。その後、同様に最大プーリングで56x56x128に変換した後、畳み込み層3層で56x56x256のデータに変換します。続いて、同様に28x28x512のデータに変換し、また同様に14x14x512のデータに変換しています。この後、最大プーリングを適用して、7x7x512に変換しています。つまり、画像全体をこの時点で7x7の領域に分割して、それぞれの領域に512次元のベクトルが計算されている、ということになります。そのあと、2層のFC層+ReLUを用いて、1x1x4096のベクトルに変換し、1層のFCを用いてクラス分類(1000クラスのクラス分類)のための1x1x1000次元のベクトルに変換したあと、ソフトマックスを適用しています。各層のデータサイズをわかりやすく並べると次のようになります。

- 0: 224x224x3 (入力)
- 1: 224x224x64 (Conv)
- 2: 224x224x64 (Conv)
- 3: 112x112x64 (Max Pooling)
- 4: 112x112x128 (Conv)
- 5: 112x112x128 (Conv)
- 6: 56x56x128 (Max Pooling)
- 7: 56x56x256 (Conv)
- 8: 56x56x256 (Conv)

9: 56x56x256 (Conv)  
10: 28x28x256 (Max Pooling)  
11: 28x28x512 (Conv)  
12: 28x28x512 (Conv)  
13: 28x28x512 (Conv)  
14: 14x14x512 (Max Pooling)  
15: 14x14x512 (Conv)  
16: 14x14x512 (Conv)  
17: 14x14x512 (Conv)  
18: 7x7x512 (Max Pooling)  
19: 1x1x4096 (FC)  
20: 1x1x4096 (FC)  
21: 1x1x1000 (FC)  
22: 1x1x1000 (Softmax)

# まとめ

- **ニューラルネットワークの学習**

- 確率的勾配降下法 (SGD)
- Momentum, AdaGrad, AdaDelta, Adam
- 過学習への対応: Weight Decay, Dropout, Batch Normalization

- **畳み込みニューラルネットワーク (CNN)**

- CNNは画像の形状／構造を捉えることに適したNN
- 畳み込み層 (Convolution Layer) とプーリング層 (Pooling Layer) を持つ
- 畳み込み層ではフィルターの学習を行い、プーリング層は画像縮退を行う。
- 畳み込み層、プーリング層のforward、backward演算を実装することでCNNを実現



32

今回のまとめです。